Enumerations in C Programming

User Role Messages and Color Enumeration

Name: Rudra Pratap Singh

SAP ID: 590032811

Batch: 16

Introduction to Enumeration

An enumeration (enum) in C is a user-defined data type that consists of a set of named integer constants. It makes a program easier to read and maintain because meaningful names are used instead of plain numbers.

Syntax:

```
enum enum_name { NAME1, NAME2, NAME3, ... };
```

Key points:

- By default, the first name is given the value 0, the next 1, and so on.
- Each name is an integer constant, so it can be used in switch statements and loops.
- C cannot print the name of an enum constant directly, so an array of strings is used for that.

Question 1: User Roles Using Enumeration

Problem Statement

Create an enumeration for user roles (ADMIN, USER, GUEST) and display a message based on the role entered.

Algorithm

1. Define an enum Role with the constants ADMIN, USER and GUEST.
2. Read the role from the user as text.
3. Convert the text to upper case and match it with the enum constants.
4. Use a switch statement to display the welcome message for the matched role.
5. If nothing matches, display Invalid Role.

Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Enumeration for user roles */
enum Role { ADMIN, USER, GUEST, INVALID };

/* Convert a text input into the matching enum value */
enum Role getRole(char input[])
{
    int i;
    for (i = 0; input[i] != '\0'; i++)
        input[i] = toupper(input[i]);

    if (strcmp(input, "ADMIN") == 0) return ADMIN;
    if (strcmp(input, "USER") == 0)  return USER;
    if (strcmp(input, "GUEST") == 0) return GUEST;
    return INVALID;
}

int main()
{
    char input[20];
    enum Role role;

    printf("Enter role (ADMIN / USER / GUEST): ");
    scanf("%19s", input);

    role = getRole(input);

    switch (role)
    {
        case ADMIN:
            printf("Welcome Admin\n");
            break;
        case USER:
            printf("Welcome User\n");
            break;
        case GUEST:
            printf("Welcome Guest\n");
            break;
        default:
            printf("Invalid Role\n");
    }
    return 0;
}
```

Input and Output

	Details
Input	GUEST
Output	Welcome Guest

Other test runs:

Input	Output
ADMIN	`Welcome Admin`
USER	`Welcome User`
xyz	`Invalid Role`

Explanation

The enum Role assigns ADMIN = 0, USER = 1 and GUEST = 2. The function getRole() converts the typed text into the matching enum value, and the switch statement selects the correct message. The extra constant INVALID handles wrong inputs.

Question 2: Enum Names and Values of 12 Colors

Problem Statement

Print all enum names and their integer values for 12 colors using a loop.

Algorithm

1. Define an enum Color with 12 colors: RED to CYAN (values 0 to 11).
2. Store the color names in an array of strings, in the same order as the enum.
3. Run a for loop from RED (0) to CYAN (11).
4. In each iteration print the name and its integer value in the form NAME=value.

Program

```
#include <stdio.h>

/* Enumeration of 12 colors (values start from 0) */
enum Color { RED, BLUE, GREEN, YELLOW, ORANGE, PURPLE,
             PINK, BROWN, BLACK, WHITE, GRAY, CYAN };

int main()
{
    /* C cannot print enum names directly, so use an array */
    const char *names[] = {
        "RED", "BLUE", "GREEN", "YELLOW",
        "ORANGE", "PURPLE", "PINK", "BROWN",
        "BLACK", "WHITE", "GRAY", "CYAN"
    };
    int i;

    for (i = RED; i <= CYAN; i++)
    {
        printf("%s=%d", names[i], i);
        if (i != CYAN)
            printf(", ");
    }
    printf("\n");
    return 0;
}
```

Input and Output

	Details
Input	`No Input`
Output	`RED=0, BLUE=1, GREEN=2, YELLOW=3, ORANGE=4, PURPLE=5, PINK=6, BROWN=7, BLACK=8, WHITE=9, GRAY=10, CYAN=11`

Explanation

Since no values are assigned manually, RED gets 0 and every next color increases by one, so CYAN gets 11. The loop variable i moves through these values, and the names array is indexed with i to print the matching name next to its number.

Conclusion

In this project, enumerations were used to give meaningful names to integer constants. The first program used an enum with a switch statement to display role based messages, and the second used a loop to print the names and integer values of 12 colors. Both programs were compiled and tested, and they produced the expected outputs.